

```

from copy import deepcopy

MOVES = {
    "Up": (-1, 0),
    "Down": (1, 0),
    "Left": (0, -1),
    "Right": (0, 1)
}

def input_state(name):
    print(f"\nEnter {name} state (use 0 for blank):")

    state = []
    for i in range(3):
        while True:
            row = list(map(int, input(f"Row {i + 1}: ").split()))

            if len(row) == 3:
                state.append(row)
                break

        print("Enter exactly 3 numbers.")

    return state

def print_state(state):
    for row in state:
        print(" ".join(map(str, row)))
    print()

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def get_neighbors(state):
    x, y = find_blank(state)

    for move, (dx, dy) in MOVES.items():
        nx = x + dx
        ny = y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = deepcopy(state)

            # Move blank
            new_state[x][y], new_state[nx][ny] = \
                new_state[nx][ny], new_state[x][y]

            yield new_state, move

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

def dfs(initial, goal):
    # Stack contains: (state, path of states, moves)
    stack = [(initial, [initial], [])]

    visited = set()

    while stack:
        state, path, moves = stack.pop()

        state_tuple = state_to_tuple(state)

        if state_tuple in visited:
            continue

```

```

        visited.add(state_tuple)

    # Goal reached
    if state == goal:
        return path, moves

    # Explore neighboring states
    for new_state, move in get_neighbors(state):
        if state_to_tuple(new_state) not in visited:
            stack.append(
                (new_state, path + [new_state], moves + [move])
            )

    return None, None

initial = input_state("initial")
goal = input_state("goal")

print("\n===== INITIAL STATE =====")
print_state(initial)

print("===== GOAL =====")
print_state(goal)

print("Solving using DFS...\n")

path, moves = dfs(initial, goal)

if path is None:
    print("No solution found.")
else:
    print("===== SOLUTION =====\n")

    for i, state in enumerate(path):
        if i == 0:
            print("Initial State:")
        else:
            print(f"Move {i}: {moves[i - 1]}")

        print_state(state)

    print("Goal reached!")
    print("Total moves:", len(moves))

```

```
4 8 5
3 2 1

Move 29513: Right
7 0 6
4 8 5
3 2 1

Move 29514: Right
7 6 0
4 8 5
3 2 1

Move 29515: Down
7 6 5
4 8 0
3 2 1

Move 29516: Left
7 6 5
4 0 8
3 2 1

Move 29517: Left
7 6 5
0 4 8
```